CS781 Capstone Project: In-House Legal Intelligence System

Samhitha Mastha
Roll Number: 240634
samhitham24@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This report documents an end-to-end legal natural-language-processing pipeline developed across seven competition phases under data and compute constraints. A compact legal foundation model was trained from scratch on a mixed corpus using a custom tokenizer, and four substantially different architectures were evaluated. Task-specific descendants and adaptation methods were then compared for legal named-entity recognition, rhetorical-role prediction, court-judgment prediction, statute identification, abstractive summarization, and prior-case retrieval. The best official scores were 0.8287 BERTScore for Phase 1, 0.4251 macro-F1 for Phase 2, 0.4752 macro-F1 for Phase 3, 0.6246 macro-F1 for Phase 4, 0.3584 macro-F1 for Phase 5, 0.8318 BERTScore for Phase 6, and 0.3904 micro-F1@7 for Phase 7. Results show that adaptation strategy and evaluation split can materially change model rankings, while compact distilled systems were competitive in several phases.

1 Competition Result

Codabench username: s_240634 **Roll number:** 240634
Code repository: https://github.com/SamhithaMastha/CS781_PROJECT

Table 1: Official phase scores and ranks, recorded on November 9, 2026.

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	BERTScore 0.8287	95/142	4
2	Legal NER	Macro-F1 0.4251	98/133	7
3	Rhetorical role prediction	Macro-F1 0.4752	83/131	4
4	Court judgment prediction	Macro-F1 0.6246	114/131	7
5	Legal statute identification	Macro-F1 0.3584	87/133	5
6	Abstractive summarization	BERTScore 0.8318	91/131	6
7	Prior Case Retrieval	Micro-F1@7 0.3904	66/131	6

The minimum requirement of four distinct submissions per phase was met. The total number of valid submissions across all phases was 39.

2 Introduction and Problem Description

The capstone investigated whether a compact, self-contained legal NLP system could support several legal-language tasks under restricted data and compute resources. The seven phases cov-

ered legal language modelling, strict-span named-entity recognition, rhetorical-role classification, judgment prediction, multi-label statute identification, abstractive summarization, and prior-case retrieval. Each task used the official competition format and its corresponding primary metric.

The central design constraints shaped the whole pipeline: the Phase 1 foundation model had to remain within the 50M–300M parameter range, pretrained language models and external legal corpora were not used for the foundation stage, and experiments had to fit practical Colab/Kaggle-style resources. Later tasks introduced additional difficulties, including exact character-offset matching for NER, long legal documents, multi-label outputs, and generation metrics such as BERTScore and ROUGE [3, 2]. The experiments therefore compared multiple compact architectures and parameter-efficient adaptation methods rather than relying on one large model.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

Phase 1 used four legal sources: Acts, High Courts, Supreme Court judgments, and Tribunals. Table 2 summarises the cleaning counts and the resulting training and validation mixture. High Courts and Tribunals were sampled at 50% in the training mixture to prevent these much larger sources from dominating the smaller sources.

Table 2: Phase 1 corpus after cleaning. Keep probability is the sampling rate in the training mixture.

Source	Docs read	Dups	Short	Train docs	Val docs	Train words	Keep prob.
Acts	18,822	90	5	4,958	78	30.0M	1.0
High Courts	2,536,738	115,596	75,154	141,757	3,335	60.0M	0.5
Supreme Court	55,528	154	87	12,523	274	50.0M	1.0
Tribunals	869,132	4,796	3,248	15,026	1,623	30.0M	0.5
Total	3,480,220	120,636	78,494	174,264	5,310	170.0M	–

For the downstream phases, the official task splits differ substantially. Phase 2 has 70 training and 35 test documents with 12 entity labels; the local development split used 63 training and 7 validation documents. Phase 3 contains two domains, Corporate Law (CL) and Information Technology (IT), with 40 training documents per domain and separate development and test documents with no shared identifiers. Phase 4 used a 3,000-case development-training subset and a 1,988-case development set, while the final test set contains 1,517 cases. Phase 5 contains 42,750 training, 10,181 development, and 13,019 test documents. Phase 6 has 7,030 training documents and 100 test documents. Phase 7 has 827 training queries against 4,320 candidates, with separate development and test pools.

3.2 Corpus statistics that actually informed design

After BPE tokenization, the Phase 1 training set contains 222.5M tokens and the validation set 6.5M tokens. The custom 32k byte-pair-encoding tokenizer produces roughly 1.24–1.41 tokens per word depending on source. A 512-word prompt averages 713 tokens, with a range of 556–1,552, which informed the prompt-length and context settings used during generation.

The downstream corpus statistics motivated task-specific design choices:

- **Phase 2:** 512-token windows with 128-token overlap produced 1,069 training, 108 validation, and 648 test windows; windows without entities were retained with probability 0.3. The 70

training documents contain 8,393 gold spans, with a median of 105 spans per document and median document length of about 26,600 characters. No gold spans overlap, nest, or touch. The oracle ceiling was 0.9053, while the WIT label scored 0 even under the oracle.

- **Phase 3:** Training contains 10,593 CL sentences and 6,252 IT sentences. Sentence-level context chunks average 45 tokens, while document-level chunks average about 960 tokens. Median sentence length is 27 words in CL and 28 in IT. Rare roles and domain imbalance were important because Dissent has no IT training examples and PrecedentOverruled has no CL examples.

Table 3: Training counts of Phase 3 rhetorical roles in Corporate Law (CL) and Information Technology (IT).

Role	CL	IT	Role	CL	IT
Fact	2,979	1,119	RatioOfTheDecision	2,856	1,445
PrecedentReliedUpon	1,305	640	ArgumentRespondent	996	352
ArgumentPetitioner	976	637	RulingByLowerCourt	438	436
Statute	367	732	Dissent	284	0
PrecedentNotReliedUpon	209	360	RulingByPresentCourt	113	194
None	53	244	Issue	17	90
PrecedentOverruled	0	3			

- **Phase 4:** The median case has 2,546 tokens and the 90th percentile is 7,679 tokens; 70% of cases are longer than one context chunk. This motivated explicit long-document handling and comparison of head, tail, and full-document context.

- **Phase 5:** The 100 statute labels are strongly imbalanced, with a median of 679 training documents per label and a range from 272 to 11,730. Each document has 3.71 labels on average, and documents average 1,751 tokens with a maximum of 60,000.

- **Phase 6:** Documents have a median length of 3,128.5 words (mean 4,368.5; maximum 139,645), and reference summaries have a median length of 638 words. Summaries are strongly extractive: 95.6% of summary words occur in the source, falling to 77.8% for bigrams and 62.6% for trigrams. This made copied source content particularly relevant to ROUGE-style evaluation.

- **Phase 7:** Development has 118 queries against 1,023 candidates, with 8.25 relevant candidates per query (minimum 1, maximum 92); the test set has 237 queries against 1,727 candidates. These statistics motivated comparison of dense retrieval against a BM25 baseline and sparse–dense fusion.

3.3 Preprocessing

Phase 1 used exact-duplicate removal, minimum-length filtering, source sampling, and a custom 32k BPE tokenizer. For Phase 2, documents were converted into overlapping windows while preserving the original character positions required for strict span evaluation. Phase 3 used sentence-level and document-level context chunks, allowing the effect of broader rhetorical context to be tested. Long documents in Phases 4–6 required task-specific truncation or context selection, while Phase 7 encoded queries and candidate cases for dense retrieval and compared them with sparse BM25 scoring. These different pipelines were necessary because a single preprocessing strategy would not preserve strict offsets, long-document context, and retrieval structure simultaneously.

4 Approach and Model Evolution

4.1 Phase-1 foundation model

Four architectures were trained and compared. Model A was a decoder-only Transformer with 109.5M parameters, 12 layers, hidden size 768, and 12 attention heads. Model B was a 123.7M encoder–decoder with six encoder and six decoder layers. Model C was a 114.3M hybrid using gated linear recurrence with attention every third layer. Model D was an 85.9M efficient architecture using sliding-window attention of size 256, global attention every fourth layer, and grouped-query attention with four KV heads. All four configurations satisfied the required parameter range.

Training used AdamW, a peak learning rate of 6×10^{-4}, cosine decay, 200 warmup steps, block length 1024, and fp16. Each model was trained for 1,000 steps of 32,768 tokens, or roughly 32.8M consumed tokens. This was substantially less than one pass over the complete mixed corpus. Base generation used temperature 0.7, top-p 0.9, top-k 50, and a repetition penalty of 1.1; a decoding sweep showed that greedy decoding with no-repeat 3-grams improved local ROUGE-L but changed the BLEU trade-off.

4.2 Transfer protocol (Phases 2–7)

The later phases were implemented as task-specific descendants and comparisons across the four Phase 1 model families. The experiments deliberately varied adaptation methods, including full fine-tuning, LoRA, LoRMA, IA^3, BitFit, QLoRA, knowledge distillation, and adapters. This design made it possible to compare not only architectures but also the amount and type of trainable adaptation.

4.3 Phase 2 - Legal NER

Phase 2 compared full fine-tuning of A, LoRMA on B, LoRA on C, and knowledge distillation into D. Because the task used strict span matching, preprocessing preserved original character offsets while 512-token windows with 128-token overlap provided local context. The best official system was an ensemble based on A checkpoints. Local error accounting for one major A run recorded 604 exact matches, 273 boundary errors, 71 label confusions, 311 false positives, and 114 missed entities, illustrating why boundary reconstruction rather than label prediction alone was a major source of difficulty.

4.4 Phase 3 - Rhetorical role prediction

Phase 3 used LoRA on A, LoRMA on B, IA^3 on C, and a distilled D student, with a full A model retained as teacher. The task was evaluated across Corporate Law and Information Technology, so the experiments compared sentence-level context with broader document-level chunks and included positional information to preserve sentence order. The teacher reached a best local macro-F1 of 0.5175 at epoch 3; the later drop despite continued training-loss reduction indicated overfitting. The main submitted systems used the V1 four-epoch configuration, while additional rsLoRA and longer-training variants were retained as controlled development comparisons.

4.5 Phase 4 - Court judgment prediction

Phase 4 compared attention-pooled full fine-tuning of B, BitFit on C, full fine-tuning and distillation variants of D, and LoRMA on A. Since many cases exceeded a single context chunk, development experiments explicitly compared head, tail, and full-document context before selecting the final

strategy. The final best B system was trained on the full 37,387-case training set for three epochs with a reported threshold of 0.35. Long-document representation, pooling, and threshold selection were therefore central design choices.

4.6 Phase 5 - Legal statute identification

Phase 5 treated statute identification as multi-label prediction and compared full fine-tuning, LoRMA, QLoRA, and knowledge distillation. The 100 statute labels were highly imbalanced and documents could contain several statutes, so the experiments also compared thresholding strategies rather than relying only on a single global decision rule. The distilled D system was the strongest official submission, while the B LoRMA system was substantially weaker than the other main variants.

4.7 Phase 6 - Abstractive summarization

Phase 6 compared LoRA on A, two adapter variants on B, LoRMA on C, full fine-tuning of D, and a distilled D student. Generation quality was assessed with BERTScore and ROUGE-style metrics. Because the reference summaries were largely extractive at the word level, generation settings were also checked for their effect on output length and overlap with source content. The distilled D student achieved the strongest official BERTScore.

4.8 Phase 7 - Prior Case Retrieval (PCR)

Phase 7 compared sparse BM25, several dense retrieval variants, and a fused ensemble. Dense systems included partial fine-tuning, LoRMA, IA^3, and a distillation student, with fusion weights tuned on the development split. The development experiments showed that BM25 fusion substantially improved dense-only retrieval, making the sparse baseline a central component of the final comparison rather than merely a baseline to beat.

5 Experiments and Implementation Details

5.1 Compute and training protocol

Experiments were run using notebook-based GPU compute, including an NVIDIA Tesla T4 GPU with 16 GB memory, with checkpointing and separate runs for competing adaptation strategies. Phase 1 used mixed precision and a fixed 1,000-step budget. Later phases selected checkpoints using local task metrics where available. Wall-clock and memory measurements were explicitly recorded for Phase 3: for example, B-LoRMA used 2.39M trainable parameters (3.4%) and 1.55GB peak memory, while C-IA^3 used 0.87M trainable parameters (0.75%) and 3.83GB peak memory.

Table 4: Available final-model implementation details. Cells marked "–" were not consistently retained in the uploaded experimental records and should be completed from run logs if required.

Hyperparameter	P1	P2	P3	P4	P5	P6	P7
Representative parameters (M)	109.5	109.6	112.9	67.1	55.3	85.9	109.5
Context/block length	1024	512 + overlap	1024	6×512	2×512	1280 + 256	4×512
Peak LR	$6e{-4}$	$1e{-4}/1e{-3}$	$4e{-4}/1e{-3}$	$1e{-4}/1e{-3}$	$1e{-4}/1e{-3}$	$3e{-4}$	$5e{-5}$
Epochs/steps	1000 steps	best of 5 ep	4	3	3	600 steps	600 steps
Main technique	pretrain	FT/PEFT/KD	LoRA	pooling/KD	multi-label/KD	generation/KD	dense+BM

5.2 Local evaluation

Local validation was used to compare variants before submission. However, hidden-test and local rankings did not always agree, and Phase 3 used two local metrics that should not be conflated. The training-log metric reports E4 (D distillation) at 0.4759 and E1 (A-LoRA) at 0.4588, whereas the notebook's separate official-style 13-label metric gives E1 0.4235 and E4 0.4027. On Codabench, E1 was best at 0.4752 versus 0.4326 for E4. The very small Phase 2 validation split also made local model selection sensitive to individual documents.

5.3 Ablations and controlled comparisons

The Phase 1 decoding sweep was a controlled comparison of generation settings. Later phases compared multiple adaptation mechanisms across the four backbone families. The project also included explicit controls, such as the Phase 3 D model without distillation, allowing the effect of distillation to be measured locally.

6 Results

6.1 Phase 1

Table 5 compares the four Phase 1 architectures using validation loss, local generation scores, and official hidden-test scores. Validation was logged only at steps 500 and 1,000.

Table 5: Phase 1 results. Local scores use 300 prompts; official scores use 1,000 hidden prompts.

Model	Loss 500	Loss 1000	PPL	Local BERT/ROUGE-L	Local BLEU	Official BERT/ROUGE-L	Official BLEU
A decoder	3.781	3.370	29.1	0.8304 / 0.2076	0.0819	0.8287 / 0.1838	0.0290
C hybrid	3.780	3.388	29.6	0.8295 / 0.2046	0.0803	0.8278 / 0.1824	0.0280
D efficient	3.776	3.387	29.6	0.8284 / 0.2061	0.0807	0.8272 / 0.1812	0.0266
B encdec	4.431	4.012	55.2	0.8131 / 0.1733	0.0327	0.8166 / 0.1739	0.0180

The decoder model ranked first on all official metrics. Local and official rankings broadly agree, although absolute BLEU and ROUGE values differ because local evaluation and the hidden benchmark use different evaluation data.

6.2 Phase 2

Table 6 gives validation and official strict macro-F1. The A-checkpoint ensemble was the strongest official system.

Table 6: Phase 2 validation and official strict macro-F1.

Run	Validation	Official
Ensemble of A checkpoints	0.3551	0.4251
exp1 (A, full)	0.3519	0.4177
exp4 (D, distillation)	0.3162	0.3745
exp3b (C, LoRA, wide)	0.2934	0.3458
exp3 (C, LoRA)	0.2708	0.2992
exp2d (B, LoRMA, long)	0.2125	0.2418
exp2 (B, LoRMA)	0.1061	0.1092

Development error counts further show that boundary errors and false positives were important failure modes under strict span matching.

Table 7: Phase 2 error types on the development split.

Run	Exact	Boundary	Label confusion	False positive	Missed
exp1 (A, full)	604	273	71	311	114
exp2 (B, LoRMA)	286	553	167	1,814	56
exp3 (C, LoRA)	499	403	103	658	57
exp4 (D, distillation)	495	317	89	164	161

6.3 Phase 3

Table 8 reports the training-log development macro-F1 for all documents and both domains, together with official results where submitted. These development values are distinct from the notebook's separate official-style 13-label metric. The full A teacher was strongest locally, while A-LoRA was the best official submission.

Table 8: Phase 3 development macro-F1 (all, CL, IT) and official macro-F1.

Run	Model and method	All	CL	IT	Official
E0	A, full (teacher)	0.5175	0.4165	0.5317	–
E4	D, distillation	0.4759	0.3676	0.4878	0.4326
E1	A, LoRA	0.4588	0.4303	0.4370	0.4752
E5	D, full, no distillation	0.4514	0.3696	0.4447	–
E3	C, IA^3	0.4319	0.3238	0.4416	0.4065
E2	B, LoRMA	0.3010	0.2846	0.2911	0.2820

The controlled ablations show that document context was substantially more useful than sentence-only context, while positional encoding had a smaller effect. The rsLoRA variant scored better locally than the submitted E1 but was not submitted.

Table 9: Phase 3 controlled ablations on development macro-F1.

Run	Change from E1	Development
E6	Sentence context only, no position	0.3246
E7	Document context, no position	0.4464
E1	Document context with position (submitted)	0.4588
E8	rsLoRA scaling	0.4693

6.4 Phase 4

Table 10 gives development and official macro-F1. The final BEST_B system achieved the strongest official score. Its development threshold was selected on development data, so that local value should be interpreted as an optimistic model-selection estimate. At the default threshold of 0.5, BEST B scored 0.5955 on development; the stored best value of 0.6513 used threshold 0.35, while the best epoch-level threshold was 0.47 with a score of 0.5986.

Table 10: Phase 4 development and official macro-F1.

Run	Model and method	Development	Official
BEST B	B, full, all data, 3 epochs	0.6513	0.6246
B full	B, full, attention pooling	0.5912	0.5874
D kd	D, distillation	0.5845	0.5451
Ensemble	Five runs	0.5825	0.5420
C bitfit	C, BitFit	0.5703	0.5385
D full	D, full	0.5734	0.5364
A lorma	A, LoRMA	0.5672	0.5306

6.5 Phase 5

The distilled D system was the best official Phase 5 submission, narrowly ahead of full fine-tuning and QLoRA.

Table 11: Phase 5 development macro-F1 and official macro-F1.

Run	Model and method	Development	Official
exp4	D, distillation from exp1	0.3677	0.3584
exp1	A, full	0.3632	0.3555
exp3	C, QLoRA	0.3630	0.3557
Ablation	A, LoRMA, head mode	0.3449	0.3382
exp2	B, LoRMA	0.2360	0.2344

The best run also obtained micro-F1 0.4552 and subset accuracy 0.0326. Per-label thresholds improved development macro-F1 for exp1 from 0.3545 with a single global threshold to 0.3632.

6.6 Phase 6

The distilled D student was strongest both locally and officially. Local development scores are shown first to make the comparison with the hidden-test ranking explicit.

Table 12: Phase 6 local development scores.

Run	BERTScore	ROUGE-1	ROUGE-L
E4 (D, distillation)	0.8349	0.3192	0.1759
E1 (A, LoRA)	0.8313	0.3054	0.1705
E3 (C, LoRMA)	0.8305	0.2809	0.1599
E6 (D, full)	0.8297	0.2918	0.1626
E2 (B, Houlsby)	0.8216	0.2415	0.1461
E5 (B, Pfeiffer)	0.8195	0.2534	0.1524

Table 13 summarises the official BERTScore comparison; its best run also achieved ROUGE-1 0.3305 and ROUGE-L 0.1771.

Table 13: Phase 6 official Codabench BERTScore F1.

Run	Official BERTScore
E4 (distillation)	0.8318
E3 (LoRMA)	0.8283
E6 (full, D)	0.8278
E1 (LoRA)	0.8271
E2 (Houlsby)	0.8183
E5 (Pfeiffer)	0.8162

A generation-length study on E1 varied the minimum new-token setting (60, 100, and 150). Local ROUGE-1 changed only from 0.3083 to 0.3088, indicating limited sensitivity of the local scores to this setting.

6.7 Phase 7

Table 14 compares dense-only development retrieval with BM25 fusion and the official Micro-F1@7. BM25 fusion substantially improved every dense system on development, but four submitted systems tied with BM25 at the best official score.

Table 14: Phase 7 development Micro-F1@7 and official Micro-F1@7.

System	Dense only	Fused BM25	Fusion weight	Official
BM25	–	0.3780	–	0.3904
Partial fine-tuning	0.2902	0.3991	0.45	0.3904
LoRMA, rank 8	0.2946	0.3924	0.40	0.3881
IA^3	0.2446	0.3880	0.30	0.3904
Distillation student	0.2535	0.3924	0.55	0.3816
Ensemble of all	0.2946	0.3980	0.40	0.3904

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

The clearest quantitative failure evidence came from Phase 2, where boundary errors and false positives were common under strict span evaluation. Rare entity labels were also substantially less frequent than DATE and STAT, creating an additional class-imbalance problem. One local pipeline issue gave WIT a score of zero, illustrating the importance of validating the evaluation and offset-reconstruction pipeline separately from model quality.

Phase 3 showed a generalization mismatch: the strongest local student was not the strongest hidden-test system. This warns against selecting a final system solely from a small development split. In Phase 4, comparisons also mixed different training budgets, so raw score differences should not automatically be interpreted as architecture effects.

7.2 What the metrics do not measure

ROUGE-L and BERTScore measure similarity to reference summaries rather than legal correctness or factual safety [2, 3]. Likewise, strict span-F1 heavily penalizes small offset mistakes even when an entity type is semantically plausible. For deployment in a legal setting, aggregate benchmark

scores should therefore be complemented by manual review, factuality checks, and error categories that reflect the cost of different mistakes.

7.3 Limitations and what you would do next

The main limitations were restricted compute, limited training budgets, short or noisy local validation splits, and incomplete long-document context. A longer controlled pretraining schedule, larger and more stable development sets where permitted, and consistent training budgets across ablations would improve causal comparisons. Retrieval should also be evaluated with additional relevance analysis because the Phase 7 tie with BM25 suggests that the current dense models may not add useful complementary information on the reported metric.

7.4 Surprising findings

Two findings were particularly notable. First, model rankings could reverse between local and hidden evaluation, as in Phase 3. Second, compact distilled systems were competitive and occasionally superior to their teachers or larger alternatives, particularly in Phases 5 and 6. Conversely, Phase 7 showed that sophisticated learned retrieval did not clearly outperform the sparse BM25 baseline on the official primary score.

8 Conclusion

This project built and evaluated a compact legal NLP pipeline across seven tasks using four Phase 1 model families and multiple task-adaptation methods. The strongest Phase 1 model was the decoder-only architecture, while later phases showed that no single adaptation strategy dominated universally. Distillation was especially competitive in statute identification and summarization, whereas sparse retrieval remained difficult to beat in prior-case retrieval. The main practical lesson was that careful evaluation matters as much as model complexity: strict alignment, small validation splits, changing training budgets, and differences between local and hidden evaluation can all change the apparent ranking of systems.

LLM Usage Declaration

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report?

Answer: ChatGPT

Q2. For each row, mark Yes or No and state the purpose in one short phrase.

Part	Used LLM?	Purpose
Understanding concepts / papers	Yes	Clarifying selected concepts during learning and report preparation.
Ideation / brainstorming at a high level	Yes	Discussing possible approaches, experiment directions, and project-level design choices across the competition phases.
Debugging small ancillary snippets	Yes	Assistance with minor code/environment questions during experimentation.
Plotting / visualization code	No	No LLM-generated plotting code was used as a final contribution.
Grammar / writing style of the report	Yes	Improving grammar, clarity, and conciseness.
Report structure / section wording	Yes	Filling the required template structure and improving section wording.
Designing the model architecture	No	Architecture choices and experimental setups were based on the student's work and competition requirements.
Hyperparameter search / tuning	No	Hyperparameter choices and experiments were performed by the student.
Analysis of results / conclusions	Yes	Help organizing and wording observations from results supplied by the student; final experimental results were retained as produced.
Writing the core training / inference code	No	Core implementation and competition code were developed and run by the student.
Other (specify)	Yes	Formatting LaTeX tables and consolidating results already obtained by the student.

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them.

Answer: No.

Q4. Apart from any permitted LLM use, what was your contribution? In 8–12 sentences describe your own understanding, architecture decisions, implementation, experiments, and analysis.

Answer: I implemented and ran the experimental pipelines for the seven phases of the competition. I worked with four Phase 1 model families and used different adaptation methods across the downstream tasks. I prepared the training and inference code, generated predictions, and submitted the resulting systems to Codabench. I selected and compared approaches including full fine-tuning, LoRA/LoRMA-style adaptation, IA^3, adapters, distillation, and retrieval fusion according to the task requirements. I monitored local validation results and compared them with the official hidden-test results. I collected the run outputs, scores, ranks, and error information used in this report. I made the experimental decisions and carried out the actual model runs and submissions. I also reviewed cases where local and official rankings differed and used those observations when interpreting the results. The final report consolidates my experimental work and results in the required template. LLM assistance was limited to the kinds of support declared above and did not replace the underlying implementation or execution of the experiments.

References

[1] A. Vaswani et al., "Attention Is All You Need," *NeurIPS*, 2017.

[2] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," 2004.

[3] T. Zhang et al., "BERTScore: Evaluating Text Generation with BERT," *ICLR*, 2020.